

ЛР6 (3 пары / 6 часов)

«Удобство для пользователя: поиск, фильтр по статусу, сортировка. Защита от ошибок»

Что уже умеем

- CRUD: добавить / список / изменить статус / удалить / редактировать
- Сохранение в `data/tasks.json`

Зачем ЛР6

Когда задач много, список неудобно листать. Мы добавим:

- **поиск** по названию (Title)
 - **фильтр** по статусу (New/InProgress/Done/All)
 - **сортировку** (например, по Id или по статусу)
 - чуть лучше сообщения и подсказки
-

0) Итог ЛР6 (что должно работать)

- ✓ Пункт меню "Поиск" — вводишь слово → показываются подходящие задачи
 - ✓ Пункт меню "Фильтр по статусу" — показываются только New/InProgress/Done (или All)
 - ✓ Пункт меню "Сортировка" — список выводится отсортированным
 - ✓ Если ничего не найдено — программа пишет "Ничего не найдено"
 - ✓ Данные в файле не ломаются (мы только показываем по-другому)
-

1) Теория (важное, по делу)

1.1. Разница между “данные” и “отображение”

- Данные (истина): список задач в `TaskService`
- Отображение: как мы их показываем пользователю (можно фильтровать/сортировать)

Важно: фильтр и поиск **не должны удалять задачи** — они только показывают.

1.2. Поиск (contains)

Поиск по части строки:

- пользователь ввёл: “тет”
- нашли: “тетрадь”, “тетя”, “кто-то”

Мы делаем поиск:

- без учёта регистра (A = a)

1.3. Фильтр по статусу

Если выбран `Done`, показываем только задачи со статусом `Done`.

1.4. Сортировка

Сортировка — это порядок вывода.

Простой вариант для новичков:

- по Id (по возрастанию)
- или по статусу (New → InProgress → Done)

2) Практика. Пара 1: добавляем методы поиска/фильтра/сортировки в Core

Шаг 1. Открыть TaskService

Visual Studio → `TaskTracker.Core → Services → TaskService.cs`

Шаг 2. Добавить метод SearchByTitle

Внутри `TaskService` добавьте:

```
public List<TaskItem> SearchByTitle(string query)
{
    query ??= "";
    query = query.Trim();

    if (query.Length == 0)
        return GetAll();

    return _tasks
        .Where(t => (t.Title ?? "").Contains(query, StringComparison.OrdinalIgnoreCase))
        .ToList();
}
```

Если подсвечивается `Where/ToList` — добавьте сверху файла:

```
using System.Linq;
```

Шаг 3. Добавить метод FilterByStatus

Добавьте:

```
public List<TaskItem> FilterByStatus(TaskStatus? status)
{
    if (status is null)
        return GetAll(); // null = All

    return _tasks.Where(t => t.Status == status).ToList();
}
```

Шаг 4. Добавить сортировки (2 простых)

4.1. Сортировка по Id

Добавьте:

```
public List<TaskItem> SortById(bool ascending = true)
{
    return ascending
        ? _tasks.OrderBy(t => t.Id).ToList()
        : _tasks.OrderByDescending(t => t.Id).ToList();
}
```

4.2. Сортировка по статусу (New → InProgress → Done)

Добавьте:

```
public List<TaskItem> SortByStatusThenId()
{
    return _tasks
        .OrderBy(t => t.Status)
        .ThenBy(t => t.Id)
        .ToList();
}
```

Шаг 5. Сборка

Build → Build Solution → Build succeeded.

Шаг 6. Коммит №1 (Core: search/filter/sort)

Git Changes → сообщение:

- feat: add search, filter and sort methods to TaskService

Commit All → Push.

3) Практика. Пара 2: добавляем пункты меню в App и вывод "через одну функцию"

Чтобы не копировать вывод списка в 10 мест, сделаем одну функцию "показать список".

Шаг 7. Открыть Program.cs

TaskTracker.App → Program.cs

Шаг 8. Сделать функцию печати списка задач

Сразу после вашей `TryReadInt` добавьте:

```
static void PrintTasks(List<TaskItem> tasks)
{
    if (tasks.Count == 0)
    {
        Console.WriteLine("Ничего не найдено.");
        return;
    }

    Console.WriteLine("Список задач:");
    foreach (var t in tasks)
    {
        Console.WriteLine($"{t.Id}. {t.Title} [{t.Status}]");
        if (!string.IsNullOrEmpty(t.Description))
            Console.WriteLine($"    Описание: {t.Description}");
    }
}
```

Убедитесь, что вверху `Program.cs` есть:

```
using TaskTracker.Core.Models;
```

Шаг 9. В пункте "2) Показать список" использовать PrintTasks

Найдите обработчик `input == "2"` и замените вывод на:

```
var tasks = service.GetAll();
if (tasks.Count == 0)
{
    Console.WriteLine("Список задач пуст.");
    continue;
}

PrintTasks(tasks);
continue;
```

(Можно оставить "Список пуст" отдельно, а можно сразу PrintTasks — как вам удобнее.)

Шаг 10. Добавить пункты меню: 6, 7, 8

В меню добавьте:

- 6) Поиск по названию
- 7) Фильтр по статусу
- 8) Сортировка списка

Шаг 11. Реализовать пункт 6: поиск

Добавьте блок:

```
if (input == "6")
{
```

```

    Console.Write("Введите текст для поиска: ");
    var query = Console.ReadLine() ?? "";

    var found = service.SearchByTitle(query);
    PrintTasks(found);
    continue;
}

```

Шаг 12. Реализовать пункт 7: фильтр по статусу

Добавьте блок:

```

if (input == "7")
{
    Console.WriteLine("Выберите статус для фильтра:");
    Console.WriteLine("0 - All (Показать всё)");
    Console.WriteLine("1 - New");
    Console.WriteLine("2 - InProgress");
    Console.WriteLine("3 - Done");

    if (!TryReadInt("Введите вариант (0/1/2/3): ", out var option))
    {
        Console.WriteLine("Ошибка: нужно число 0/1/2/3.");
        continue;
    }

    TaskStatus? status = option switch
    {
        0 => (TaskStatus?)null,
        1 => TaskStatus.New,
        2 => TaskStatus.InProgress,
        3 => TaskStatus.Done,
        _ => (TaskStatus?)null
    };
}

```

```

    if (option < 0 || option > 3)
    {
        Console.WriteLine("Ошибка: выберите 0, 1, 2 или 3.");
        continue;
    }

    var filtered = service.FilterByStatus(status);
    PrintTasks(filtered);
    continue;
}

```

Шаг 13. Реализовать пункт 8: сортировка

Добавьте блок:

```

if (input == "8")
{
    Console.WriteLine("Выберите сортировку:");
    Console.WriteLine("1 - по Id (по возрастанию)");
    Console.WriteLine("2 - по Id (по убыванию)");
    Console.WriteLine("3 - по статусу, затем по Id");

    if (!TryReadInt("Введите вариант (1/2/3): ", out var option))
    {
        Console.WriteLine("Ошибка: нужно число 1/2/3.");
        continue;
    }

    List<TaskItem> sorted;

    if (option == 1) sorted = service.SortById(true);
    else if (option == 2) sorted = service.SortById(false);
    else if (option == 3) sorted = service.SortByStatusThenId

```



```
();  
    else  
    {  
        Console.WriteLine("Ошибка: выберите 1, 2 или 3.");  
        continue;  
    }  
  
    PrintTasks(sorted);  
    continue;  
}
```

Шаг 14. Проверка (F5) — сценарий

Запустите (F5), сделайте так:

1. Добавьте 3 задачи:
 - "Купить тетрадь" (New)
 - "Сдать ЛР" (Done) — поменяйте статус
 - "Позвонить" (InProgress) — поменяйте статус
1. Поиск: введите "ку" → должна найтись "Купить..."
2. Фильтр Done → покажет только Done
3. Сортировка по статусу → New, InProgress, Done

Шаг 15. Коммит №2 (App: меню + поиск/фильтр/сортировка)

Сообщение:

- feat: add search, filter and sort commands in console app

Commit All → Push.

4) Практика. Пара 3: сохраняем удобство и пишем отчёт

Шаг 16. Очень полезное улучшение: подсказка в конце меню

В конце вывода меню добавьте строку:

```
Console.WriteLine("Подсказка: 6 – поиск, 7 – фильтр, 8 – сортировка");
```

Шаг 17. Отчёт `docs/LR6_Report.md`

Создайте `docs/LR6_Report.md`:

```
# Отчёт ЛР6 – TaskTracker

## Что сделано
- В TaskService добавлены методы:
  - SearchByTitle(query)
  - FilterByStatus(status or All)
  - SortById(asc/desc)
  - SortByStatusThenId()
- В консольном приложении добавлены пункты меню:
  - 6) Поиск
  - 7) Фильтр по статусу
  - 8) Сортировка
- Добавлена функция PrintTasks для красивого вывода списка

## Как проверить
1) Добавить 3 задачи
2) Изменить статусы, чтобы были New/InProgress/Done
3) Поиск по части названия
```

- 4) Фильтр по статусу
- 5) Сортировка (по Id и по статусу)

Шаг 18. Коммит №3 (отчёт)

Сообщение:

- `docs: add LR6 report`

Commit All → Push.

5) Что сдавать (чек-лист)

- ✓ ссылка на репозиторий
- ✓ `docs/LR6_Report.md`
- ✓ видео/скрин: поиск + фильтр + сортировка
- ✓ минимум 3 коммита за ЛР6

6) Оценивание

"3" — работает поиск ИЛИ фильтр, есть отчёт

"4" — работают поиск + фильтр + сортировка, нет падений

"5" — аккуратный вывод, хорошие подсказки, проверка ошибок ввода везде